KARKAII SCHOOL ERP · BACKEND

What we built: accounts, schools & permissions

A multi-school backend where one person has one login, can belong to many schools with a different role in each, and every action is checked against that school's permissions. This document summarises the design, the database, the API, the security model and the evidence that it works.

Status as of 30 October 2026 · Node.js 22 · Express 5 · TypeScript · Prisma · PostgreSQL
Repository: **CreatixHub/backend** · run the live demo with `./demo.sh up` then `./demo.sh api-demo`

1. At a glance

20
API endpoints (sign up, sign in, schools, members, roles, permissions)

9
database tables, created by a versioned migration

110
permissions (11 modules × 10 actions)

12
default roles every new school receives

60 / 60
end-to-end checks passing (isolation, roles, sessions)

~1,260
lines of TypeScript in 28 files

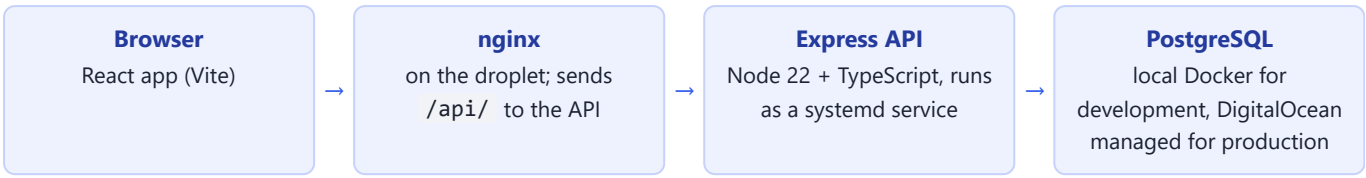
What it does, in plain words

- **Sign up and sign in.** A person creates one account (name, e-mail, password) and signs in. Sessions stay alive safely without asking for the password again.
- **Create a school.** Any signed-in person can create a school and becomes its *owner*. The school immediately gets 12 ready-made roles (administrator, teacher, accountant, receptionist, student, parent, librarian, warden, …).
- **One person, many schools.** The same account can be the owner of one school and a teacher in another. Switching school is one call, and the powers change with it.
- **People and permissions.** Owners and administrators add people to the school, change their roles, suspend or remove them, and can create custom roles. Every request is checked against the caller's permissions *in that school*.
- **Strict separation.** One school can never see or change another school's people or data — this is tested.

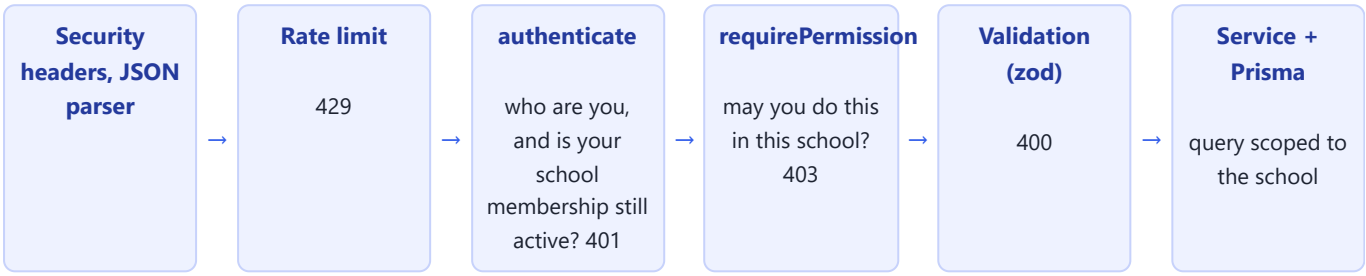
Status

Area	Status	Notes
Database schema and migration	**Done**	9 tables; applied to the local PostgreSQL
Sign up, sign in, refresh, logout, `/me`	**Done**	argon2 passwords, short-lived token + rotating httpOnly refresh cookie
Schools: create, list, switch, details	**Done**	creation is one all-or-nothing transaction
Roles, permissions, members	**Done**	110 permissions, 12 default roles, custom roles, owner protections
Automated checks	**60 / 60**	`scripts/check-all.sh` runs them against a live API
Deployment pipeline (GitHub Actions to the DigitalOcean droplet)	**Ready, waiting for the production database URL**	droplet, service and nginx route are configured; first deploy needs `DATABASE_URL` on the server
React frontend connected to this API	**Next**	the frontend already has the auth plumbing; sign-up, create-school pages and permission-based UI are the planned work
Forgot password, e-mail verification, 2-factor	**Planned**	needs an e-mail provider

2. Architecture



Inside the API, every request travels the same pipeline, so security rules cannot be skipped by forgetting them on one route:



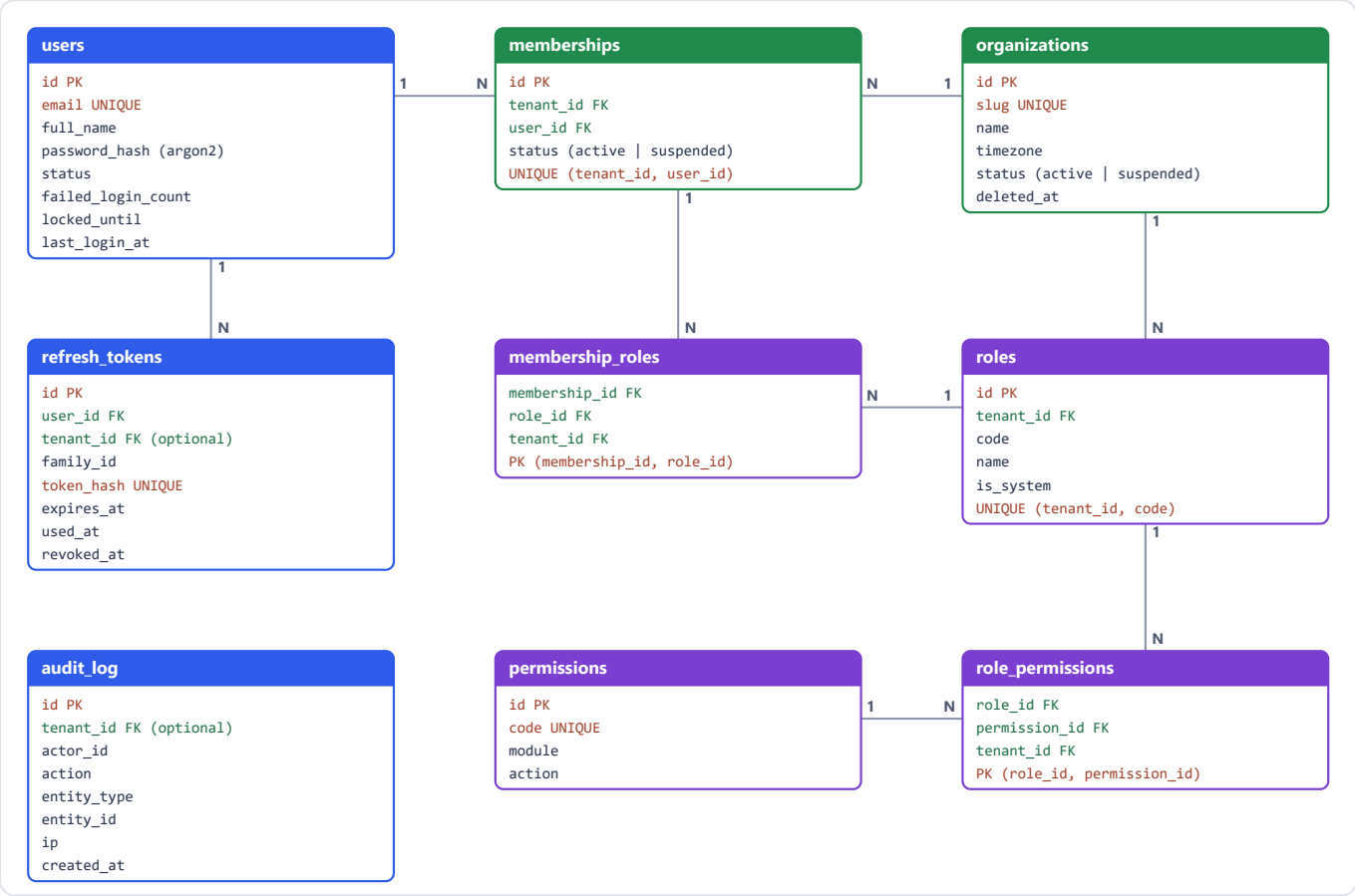
Protected by default: only health, sign up, sign in, refresh and logout are registered before the `authenticate` middleware. Anything added later is automatically behind the login check.

Code layout

Folder	Contents
`src/app.ts` , `server.ts`	builds the app (middleware order is the security design) and starts it
`src/middleware/`	`authenticate` , `requirePermission` , `errorHandler`
`src/modules/auth`	sign up, sign in, refresh, logout, `/me` , switch school; token and cookie handling (`session.ts`)
`src/modules/organizations`	create a school (transaction), list my schools, current school
`src/modules/members` , `roles`	people in a school; roles and the permission catalogue
`src/permissions.ts`	the 110 permissions (generated) and the 12 default roles (data)
`prisma/`	`schema.prisma` and the migration history
`scripts/`	live demo (`demo-api.sh`) and the 60 checks (`check-all.sh`)

3. The database

Nine tables. Colour shows the purpose: identity & sessions · schools & membership · roles & permissions



PK = primary key, FK = foreign key (reference to another table), UNIQUE = the database refuses duplicates. Lines show one-to-many relationships (1 → N). `refresh_tokens.tenant_id` and `audit_log.tenant_id` also reference `organizations` (not drawn).

Table	Purpose
`users`	One row per **person**, shared by all their schools. E-mail is unique across the system; only an argon2 hash of the password is stored.
`organizations`	One row per school (the tenant). Has a unique short name (`slug`) and a status that lets a school be suspended.
`memberships`	"This person belongs to this school." One row per (school, person), with a status so a school can switch someone off without deleting their account.
`roles`	Each school has its **own copy** of every role, so schools can later customise them independently.
`permissions`	The global catalogue: `module.action` such as `students.create` (110 rows).
`role_permissions`	Which permissions each role grants.
`membership_roles`	Which roles a person has *inside one school*.
`refresh_tokens`	One row per issued session token (only a SHA-256 hash is stored) with its login "family", expiry and usage, so stolen tokens can be detected.
`audit_log`	Who did what and when (the table exists; writing events is planned).

4. The key design: one person, many schools

A correspondent may run several schools; a parent may have children in two; a teacher may work at two. They should have **one login**. So the system separates *who you are* from *what you are in a school*:

User	↔	**Membership**	→	**Roles of that membership**
one per person e-mail + password		one per person and school		what you are inside that school

One account	School	Role there
Asha	Greenfield School	owner
	Riverdale School	owner (same role in a second school)
	Sunrise School	teacher (a different role in a third)

The one rule that keeps schools apart

> The school a request acts in is **verified against the caller's membership on every request**, and then used in **every** database query. It is never accepted from the URL or the request body. A school that tries to read or change another school's people gets "not found".

How sessions work

- **Access token** (15 minutes) says who you are and which school you are acting in. On every request the database is asked again whether the person and the membership are still active, so suspending someone works *immediately*, not when the token expires.
- **Refresh token** lives in an **httpOnly cookie** that JavaScript cannot read. It works **once**: each refresh returns a new one. If an old one is replayed later, the whole login is revoked (theft detection); if it is replayed within 10 seconds (two browser tabs), nothing is revoked and the app simply retries.
- **Switching school** issues a new token for the other school; the session remembers the active school across refreshes.

5. Roles and permissions

A **permission** is `module.action`. Modules: dashboard, students, admissions, teachers, academics, attendance, examinations, finance, organization, members, roles. Actions: view, create, edit, delete, import, export, approve, print, download, manage — the same ten switches your Roles & Permissions screen already uses. 11 × 10 = **110**, generated by a loop (never typed by hand).

Default role	Permissions	Intended for
`owner`	all (wildcard)	the person who created the school; the wildcard means future permissions apply automatically
`admin`	109	runs the school day to day (everything except `organization.manage`)
`teacher`	10	dashboard, students (view), attendance (view/create/edit), academics (view), examinations (view/create/edit)
`accountant`	13	dashboard, students (view), all finance actions
`receptionist`	6	dashboard, students (view), admissions (view/create/edit)
`student`, `parent`, `librarian`, `warden`, `transport_manager`, `nurse`, `security`	2 each	dashboard and basic school details to start; each school extends them

These role names are the ones the React app already understands, so its role-based routing and permission checks work without change. Schools can also create **custom roles** from any combination of permissions; built-in roles are protected from edits.

Protections built into people management

- Only an owner can create or change an owner.
- Nobody can change or remove their own access.
- A school always keeps at least one active owner.
- Roles are looked up inside the caller's own school, so another school's roles can never be assigned.

6. The API

All routes are under `/api`. Successful answers use `{ success, data, meta }`; errors use `{ message, errors[] }` — the formats the React app already expects.

Method	Path	Who may call	What it does
GET	/health	anyone	API and database are up
POST	/auth/signup	anyone	create an account and sign in
POST	/auth/login	anyone	sign in; returns the token, the user and their schools
POST	/auth/refresh	cookie	renew the session (single-use cookie)
POST	/auth/logout	cookie	end the session everywhere
GET	/auth/me	signed in	my account, active school, role, permissions, schools
POST	/auth/switch-organization	signed in	act in another school I belong to
POST	/organizations	signed in	create a school; I become its owner
GET	/organizations	signed in	my schools with my role in each
GET / PATCH	/organizations/current	`organization.view` / `.edit`	the active school; change name or timezone
GET	/permissions	`roles.view`	the 110-permission catalogue
GET	/roles	`roles.view`	the school's roles with permissions and head-counts
POST / PATCH / DELETE	/roles, /roles/:id	`roles.create / edit / delete`	custom roles (built-in roles are protected)
GET	/members	`members.view`	people in the school (search and paging)
POST	/members	`members.create`	add a person (new or existing account) with roles
PATCH / DELETE	/members/:id	`members.edit / delete`	change roles or suspend / remove a person

7. Security model

Risk	Defence
Stolen database	Passwords are argon2id hashes; refresh tokens are stored only as SHA-256 hashes.
Guessing passwords, finding which e-mails exist	Identical error and identical timing for “unknown e-mail” and “wrong password”; sign-in and sign-up limited to 10 per minute per address; a school creation limit; a global limit.
Stolen access token	Lives 15 minutes; the person and membership are re-checked in the database on every request.
Stolen refresh cookie	httpOnly (scripts cannot read it), limited to `/api/auth`, single-use, and reuse revokes the whole login.
Tampered tokens	Only the signing algorithm we use is accepted; a missing or malformed token is a plain 401.
One school reading another’s data	Verified school in every query; “not found” instead of “forbidden” so ids cannot be probed.
Sneaking extra fields into requests	Strict input schemas reject unknown fields.
Forgetting to protect a new route	Routes are protected by default (middleware order).
Leaking internals	Unexpected errors are logged on the server and returned as a plain 500.
Secrets in git	`.env` files are git-ignored; on the server the secrets file is readable only by root and the service user; the CI deploy key is restricted to one repository and one restart command.

Known gaps (planned): HTTPS (the site is on plain HTTP so the cookie is not yet marked Secure); e-mail verification and forgot-password; two-factor sign-in; database-level row security as a second lock; writing events into the audit log.

8. Evidence that it works

The script `scripts/check-all.sh` drives the real API and database with two schools and three people, and checks 60 behaviours. All 60 pass.

Group	Checks	What is proven
Accounts and schools	13	sign up, duplicate and reserved names refused, create school, switch school, owner rights, a person with no school is refused school routes
Catalogue and roles	4	110 permissions; 12 default roles in a new school
Members	11	add an existing or a new person, duplicates refused, validation errors, search and paging, no editing yourself
One person, two schools	13	different role and different powers in each school; a teacher is refused school settings and the member list
Isolation	4	a school sees only its own people and cannot edit or delete another school's members
Immediate effect	7	role changes apply to the existing token; suspending a member kills their token at once; non-owners cannot create owners
Role management	8	create, edit, delete custom roles; built-in roles protected; unknown permissions refused

Session behaviour (refresh rotation, replay detection, logout) was verified separately: refresh returns a new token (200); replaying the old cookie immediately gives 409 and revokes nothing; replaying it later gives 401 and revokes the whole login; after logout the cookie is dead.

Run it yourself

Command (from the `karkaii` folder)	Result
`./demo.sh up`	starts PostgreSQL, the API, the app, the landing page and the database viewer
`./demo.sh api-demo`	narrated walkthrough of everything above against the live API
`backend/scripts/check-all.sh`	the 60 automated checks
`./demo.sh status` · `down` · `reset-db`	what is running; stop everything; start from an empty database

9. Deployment and what is next

Deployment

- **GitHub Actions** (`.github/workflows/deploy.yml`): on every push to `main` it type-checks, builds, uploads to the droplet over SSH, installs production dependencies, restarts the service and waits for the health check. If the new release is unhealthy it **rolls back automatically**.
- **Droplet:** a locked-down `karkaii` service user, a systemd unit that applies database migrations and loads the permission catalogue before each start, an nginx route sending `/api/` to the API, and a secrets file readable only by root and the service user.
- **Database:** PostgreSQL on DigitalOcean. The one step still pending is putting the production `DATABASE_URL` on the server. Details: `docs/DEPLOY.md`.

Next

#	Work
1	**Connect the React app:** real sign up and sign in, a create-school page, redirects for people without a school, permission-based menus, a school switcher.
2	**First production deploy** once the database URL is set; then HTTPS and a domain (cookies become Secure).
3	Forgot password and e-mail verification; invitations by e-mail.
4	The first real school module (students), copying the members module's pattern, with a test that school A cannot touch school B's students.
5	Move the 60 checks into the CI pipeline; write events into the audit log; database row-level security.